

Tarea 6: Agenda de Contactos

Semana 6: Archivos y Persistencia de Datos

NLPY-1: Fundamentos de Python

1. Introducción

En esta sexta semana del curso, exploraremos uno de los aspectos más importantes de la programación: **la persistencia de datos**. Hasta ahora, todos los programas que hemos desarrollado pierden su información cuando finalizan su ejecución. En la vida real, las aplicaciones necesitan almacenar y recuperar información de manera permanente.

El manejo de archivos es fundamental en cualquier aplicación profesional, desde sistemas de gestión empresarial hasta aplicaciones científicas que procesan grandes volúmenes de datos. Python ofrece herramientas poderosas y elegantes para trabajar con archivos de texto, archivos CSV, JSON y otros formatos.

En esta tarea desarrollarás una **Agenda de Contactos** completa que permite almacenar, buscar, modificar y eliminar información de contactos, manteniendo todos los datos guardados en archivos que persisten entre ejecuciones del programa.

1.1. Objetivos de Aprendizaje

Al completar esta tarea, el estudiante será capaz de:

- Comprender los conceptos de persistencia de datos y operaciones de archivo
- Utilizar las funciones `open()`, `read()`, `write()`, y `close()` correctamente
- Aplicar el gestor de contexto `with` para manejo seguro de archivos
- Trabajar con diferentes modos de apertura de archivos (`r`, `w`, `a`, `r+`, etc.)
- Procesar archivos CSV utilizando el módulo `csv`
- Implementar operaciones CRUD (Create, Read, Update, Delete) con archivos
- Manejar excepciones específicas de operaciones de archivo
- Validar y limpiar datos antes de guardarlos
- Organizar código en funciones especializadas para operaciones de archivo

2. Descripción del Proyecto

Desarrollarás un sistema de **Agenda de Contactos** que permite gestionar información personal y profesional de contactos. El sistema debe almacenar los datos en archivos CSV y proporcionar una interfaz de menú para realizar todas las operaciones.

2.1. Funcionalidades Requeridas

2.1.1. 1. Gestión de Contactos (40 puntos - Funcionalidad)

Tu programa debe implementar las siguientes funcionalidades:

A) Agregar Nuevo Contacto (10 puntos)

- Solicitar: nombre completo, teléfono(s), email, dirección, categoría (familia/amigo/trabajo/otro)

- Validar formato de email (debe contener @ y dominio)
- Validar que el nombre no esté vacío
- Permitir múltiples teléfonos separados por comas
- Guardar en archivo CSV automáticamente
- Confirmar la adición exitosa

B) Listar Todos los Contactos (5 puntos)

- Mostrar todos los contactos en formato tabular
- Incluir numeración para cada contacto
- Mostrar mensaje apropiado si no hay contactos
- Formato limpio y legible

C) Buscar Contactos (8 puntos)

- Búsqueda por nombre (coincidencia parcial, case-insensitive)
- Búsqueda por categoría
- Búsqueda por teléfono
- Mostrar todos los resultados encontrados
- Indicar si no se encontraron coincidencias

D) Actualizar Contacto (8 puntos)

- Buscar contacto a actualizar
- Mostrar datos actuales
- Permitir modificar cada campo (dejar vacío para no cambiar)
- Validar nuevos datos
- Actualizar el archivo CSV
- Confirmar la actualización

E) Eliminar Contacto (5 puntos)

- Buscar y mostrar contacto a eliminar
- Solicitar confirmación antes de eliminar
- Eliminar del archivo CSV
- Confirmar la eliminación

F) Exportar/Importar (4 puntos)

- Exportar contactos a archivo de texto legible
- Crear respaldo (backup) del archivo CSV
- Mostrar estadísticas (total de contactos por categoría)

2.1.2. 2. Requisitos Técnicos

Manejo de Archivos:

- Usar gestor de contexto `with` en TODAS las operaciones de archivo
- Archivo principal: `contactos.csv`
- Manejar correctamente el encoding UTF-8
- Implementar manejo de excepciones para operaciones de archivo

Formato CSV:

- Usar módulo `csv` de Python
- Incluir encabezados en el archivo
- Campos: nombre, telefonos, email, direccion, categoria, fecha_creacion

Validaciones:

- Nombre: no vacío, mínimo 3 caracteres
- Email: formato válido (contiene @ y punto)
- Teléfono: solo números, guiones y espacios
- Categoría: debe ser una de las opciones válidas

Organización del Código:

- Función para cargar contactos desde CSV
- Función para guardar contactos en CSV
- Función para cada operación CRUD
- Función de menú principal
- Funciones de validación separadas

3. Ejemplo de Implementación

A continuación se muestra un ejemplo completo del funcionamiento esperado:

3.1. Estructura de Datos

```
1 nombre,telefonos,email,direccion,categoria,fecha_creacion
2 Juan Perez,8888-8888|7777-7777,juan@email.com,San Jose Centro,trabajo
   ,2024-10-24
3 Maria Rodriguez,6666-6666,maria@email.com,Cartago,familia,2024-10-24
4 Carlos Mora,5555-5555,carlos@email.com,Heredia,amigo,2024-10-24
```

Listing 1: Ejemplo de archivo contactos.csv

3.2. Plantilla de Código

A continuación se muestra la estructura básica que debes seguir para implementar tu solución:

```
1 import csv
2 import os
3 from datetime import datetime
4
5 # Constantes
6 ARCHIVO_CONTACTOS = 'contactos.csv'
7 CATEGORIAS_VALIDAS = ['familia', 'amigo', 'trabajo', 'otro']
8 ENCABEZADOS = ['nombre', 'telefonos', 'email', 'direccion',
9                'categoria', 'fecha_creacion']
10
11 def crear_archivo_si_no_existe():
12     """Crea el archivo CSV con encabezados si no existe."""
13     # TODO: Implementar creacion de archivo con DictWriter
14     pass
15
16 def cargar_contactos():
17     """Carga todos los contactos desde el archivo CSV."""
18     # TODO: Usar DictReader para leer el archivo
19     # TODO: Manejar FileNotFoundError
20     # TODO: Retornar lista de diccionarios
21     pass
22
23 def guardar_contactos(contactos):
24     """Guarda todos los contactos en el archivo CSV."""
25     # TODO: Usar DictWriter para escribir todos los contactos
26     # TODO: Incluir encabezados
27     # TODO: Manejar excepciones
28     pass
29
30 def validar_email(email):
31     """Valida el formato basico de un email."""
32     # TODO: Verificar que contiene @ y punto en el dominio
33     pass
34
35 def validar_nombre(nombre):
36     """Valida que el nombre sea valido."""
37     # TODO: Verificar longitud minima
38     # TODO: Verificar que contiene letras
39     pass
40
41 def validar_telefono(telefono):
42     """Valida que el telefono solo contenga caracteres permitidos."""
43     # TODO: Verificar caracteres numericos y separadores
44     pass
45
46 def agregar_contacto():
47     """Agrega un nuevo contacto a la agenda."""
48     # TODO: Solicitar datos del contacto
49     # TODO: Validar cada campo
50     # TODO: Crear diccionario con los datos
51     # TODO: Agregar a la lista y guardar
52     pass
53
```

```
54 def listar_contactos():
55     """Lista todos los contactos de la agenda."""
56     # TODO: Cargar contactos
57     # TODO: Mostrar en formato tabular
58     # TODO: Manejar agenda vacía
59     pass
60
61 def buscar_contactos():
62     """Busca contactos por diferentes criterios."""
63     # TODO: Mostrar menu de busqueda
64     # TODO: Implementar busqueda por nombre
65     # TODO: Implementar busqueda por categoria
66     # TODO: Implementar busqueda por telefono
67     # TODO: Mostrar resultados
68     pass
69
70 def actualizar_contacto():
71     """Actualiza la informacion de un contacto existente."""
72     # TODO: Buscar contacto a actualizar
73     # TODO: Mostrar datos actuales
74     # TODO: Solicitar nuevos valores
75     # TODO: Validar y actualizar
76     pass
77
78 def eliminar_contacto():
79     """Elimina un contacto de la agenda."""
80     # TODO: Buscar contacto
81     # TODO: Mostrar informacion
82     # TODO: Solicitar confirmacion
83     # TODO: Eliminar y guardar
84     pass
85
86 def exportar_a_texto():
87     """Exporta todos los contactos a un archivo de texto legible."""
88     # TODO: Cargar contactos
89     # TODO: Crear archivo de texto
90     # TODO: Escribir con formato legible
91     pass
92
93 def crear_respaldo():
94     """Crea una copia de respaldo del archivo de contactos."""
95     # TODO: Leer archivo original
96     # TODO: Crear copia con timestamp
97     pass
98
99 def mostrar_estadisticas():
100     """Muestra estadisticas sobre los contactos."""
101     # TODO: Contar contactos por categoria
102     # TODO: Calcular porcentajes
103     # TODO: Mostrar informacion resumida
104     pass
105
106 def menu_principal():
107     """Muestra el menu principal y maneja la navegacion."""
108     crear_archivo_si_no_existe()
109
```

```

110 while True:
111     # TODO: Mostrar menu de opciones
112     # TODO: Leer opcion del usuario
113     # TODO: Llamar a la funcion correspondiente
114     # TODO: Manejar opcion de salir
115     pass
116
117 # Ejecutar el programa
118 if __name__ == "__main__":
119     menu_principal()

```

Listing 2: Plantilla básica del sistema

3.2.1. Ejemplo de Implementación: Función cargar_contactos()

Como guía, aquí hay UN EJEMPLO de cómo implementar la función de carga:

```

1 def cargar_contactos():
2     """Carga todos los contactos desde el archivo CSV."""
3     contactos = []
4     try:
5         with open(ARCHIVO_CONTACTOS, 'r', newline='',
6                 encoding='utf-8') as archivo:
7             lector = csv.DictReader(archivo)
8             for fila in lector:
9                 contactos.append(fila)
10        return contactos
11    except FileNotFoundError:
12        print("Archivo no encontrado. Se creara uno nuevo.")
13        crear_archivo_si_no_existe()
14        return []
15    except Exception as e:
16        print(f"Error al cargar: {e}")
17        return []

```

Listing 3: Ejemplo: Implementación de cargar_contactos

Nota importante: Este es SOLO un ejemplo de referencia para una función. El resto del código debe ser implementado por el estudiante siguiendo la plantilla.

3.3. Ejemplo de Ejecución

```

1 Bienvenido a la Agenda de Contactos
2 Desarrollado para NLPY-1 - Semana 6
3
4 =====
5 AGENDA DE CONTACTOS
6 =====
7 1. Agregar nuevo contacto
8 2. Listar todos los contactos
9 3. Buscar contactos
10 4. Actualizar contacto
11 5. Eliminar contacto
12 6. Exportar a archivo de texto
13 7. Crear respaldo
14 8. Mostrar estadísticas

```

```
15 9. Salir
16 =====
17
18 Seleccione una opcion: 1
19
20 =====
21 AGREGAR NUEVO CONTACTO
22 =====
23
24 Nombre completo: Maria Fernandez Lopez
25 Telefono(s) (separados por comas): 8888-8888, 2222-2222
26 Email: maria.fernandez@email.com
27 Direccion: Cartago, Dulce Nombre
28 Categorias disponibles: familia, amigo, trabajo, otro
29 Categoria: familia
30
31 Contacto agregado exitosamente!
32 Total de contactos: 4
33
34 =====
35 AGENDA DE CONTACTOS
36 =====
37 ...
38 Seleccione una opcion: 3
39
40 =====
41 BUSCAR CONTACTOS
42 =====
43 1. Buscar por nombre
44 2. Buscar por categoria
45 3. Buscar por telefono
46
47 Seleccione tipo de busqueda: 2
48 Categorias: familia, amigo, trabajo, otro
49 Ingrese categoria: familia
50
51 2 contacto(s) encontrado(s):
52 -----
53
54 Nombre: Maria Rodriguez
55 Telefonos: 6666-6666
56 Email: maria@email.com
57 Categoria: familia
58
59 Nombre: Maria Fernandez Lopez
60 Telefonos: 8888-8888, 2222-2222
61 Email: maria.fernandez@email.com
62 Categoria: familia
63
64 =====
65 AGENDA DE CONTACTOS
66 =====
67 ...
68 Seleccione una opcion: 8
69
```

```
70 =====
71 ESTADISTICAS DE LA AGENDA
72 =====
73
74 Total de contactos: 4
75
76 Contactos por categoria:
77     Familia: 2 (50.0%)
78     Amigo: 1 (25.0%)
79     Trabajo: 1 (25.0%)
80     Otro: 0 (0.0%)
81
82 Contacto mas antiguo: Juan Perez (2024-10-24)
```

Listing 4: Ejemplo de ejecución del programa

4. Defensa del Código (60 % de la Nota)

La defensa oral es un componente crucial de la evaluación. Debes demostrar comprensión profunda de:

4.1. Aspectos Técnicos a Dominar

1. Manejo de Archivos (15 puntos)

- ¿Qué es el gestor de contexto `with`? ¿Por qué es importante?
- Diferencias entre modos de apertura: `'r'`, `'w'`, `'a'`, `'r+'`, `'w+'`
- ¿Qué pasa si no se cierra un archivo correctamente?
- Explicar el concepto de *file handle* o descriptor de archivo
- ¿Cómo se maneja el encoding en archivos de texto?

2. Módulo CSV (10 puntos)

- ¿Por qué usar el módulo `csv` en lugar de procesar texto manualmente?
- Diferencia entre `csv.reader` y `csv.DictReader`
- ¿Qué es el parámetro `newline=''` y por qué se usa?
- Explicar `DictWriter` y el uso de `fieldnames`
- ¿Cómo se manejan caracteres especiales (comas, saltos de línea) en CSV?

3. Operaciones de Archivo (10 puntos)

- Explicar el flujo completo de agregar un contacto (desde input hasta disco)
- ¿Cómo funciona la actualización de archivos CSV? ¿Se puede editar ^{en} el lugar?
- Estrategias para búsqueda eficiente en archivos
- ¿Qué es el módulo `os` y para qué lo usamos?

- Diferencia entre rutas absolutas y relativas

4. Validación y Manejo de Errores (10 puntos)

- Excepciones específicas de archivos: `FileNotFoundError`, `PermissionError`, `IOError`
- ¿Por qué validar datos ANTES de guardarlos?
- Estrategias de validación de email, teléfono, etc.
- ¿Qué es la sanitización de datos?
- Recuperación ante errores durante escritura de archivos

5. Organización y Buenas Prácticas (15 puntos)

- Justificar la estructura del código en funciones
- ¿Por qué usar constantes para nombres de archivo y categorías?
- Explicar el patrón cargar-modificar-guardar
- Importancia de los respaldos automáticos
- Comentarios y documentación del código
- Nombres descriptivos de variables y funciones

4.2. Preguntas de Defensa Típicas

Prepárate para responder preguntas como:

- "Muéstame en tu código dónde usas el gestor de contexto `with`. ¿Qué pasaría si no lo usaras?"
- "¿Cómo maneja tu programa el caso donde el archivo de contactos no existe?"
- "Si el programa se interrumpe mientras se está guardando, ¿qué pasa con los datos?"
- ".Explica línea por línea tu función de búsqueda. ¿Por qué usaste ese enfoque?"
- "¿Qué mejoras implementarías para manejar miles de contactos?"
- "Muéstame cómo validaste el formato del email. ¿Es suficiente tu validación?"
- "¿Por qué elegiste CSV en lugar de otro formato como JSON o texto plano?"
- ".Explica cómo almacenas múltiples teléfonos en un solo campo CSV"

5. Rúbrica de Evaluación

5.1. Funcionalidad del Código (40 puntos)

Agregar Contacto (10 puntos):

- 3 puntos: Solicita y almacena todos los campos requeridos
- 3 puntos: Validaciones funcionan correctamente (email, nombre, teléfono)
- 2 puntos: Guarda correctamente en CSV con encoding UTF-8

- 2 puntos: Confirmación y manejo de errores

Listar Contactos (5 puntos):

- 2 puntos: Carga y muestra todos los contactos correctamente
- 2 puntos: Formato tabular claro y legible
- 1 punto: Maneja caso de agenda vacía

Buscar Contactos (8 puntos):

- 3 puntos: Búsqueda por nombre funciona (parcial, case-insensitive)
- 2 puntos: Búsqueda por categoría funciona
- 2 puntos: Búsqueda por teléfono funciona
- 1 punto: Mensajes apropiados cuando no hay resultados

Actualizar Contacto (8 puntos):

- 2 puntos: Encuentra y muestra contacto correctamente
- 3 puntos: Permite modificar campos con validación
- 2 puntos: Guarda cambios correctamente en CSV
- 1 punto: Maneja múltiples contactos con nombre similar

Eliminar Contacto (5 puntos):

- 2 puntos: Encuentra y muestra contacto a eliminar
- 2 puntos: Solicita confirmación y elimina correctamente
- 1 punto: Actualiza archivo CSV

Exportar/Estadísticas (4 puntos):

- 2 puntos: Exporta a texto legible correctamente
- 1 punto: Crea respaldos
- 1 punto: Muestra estadísticas por categoría

5.2. Defensa del Código (60 puntos)**Comprensión de Archivos (15 puntos):**

- 5 puntos: Explica gestor de contexto y modos de apertura
- 5 puntos: Demuestra comprensión de lectura/escritura
- 5 puntos: Entiende encoding y manejo de caracteres especiales

Módulo CSV (10 puntos):

- 5 puntos: Explica uso de DictReader y DictWriter
- 3 puntos: Comprende manejo de delimitadores y formato

- 2 puntos: Explica ventajas sobre procesamiento manual

Operaciones CRUD (10 puntos):

- 3 puntos: Explica flujo de creación de contactos
- 3 puntos: Comprende lectura y búsqueda
- 2 puntos: Explica actualización correctamente
- 2 puntos: Comprende eliminación segura

Validación y Errores (10 puntos):

- 4 puntos: Identifica y maneja excepciones de archivo
- 3 puntos: Explica validaciones implementadas
- 3 puntos: Demuestra comprensión de sanitización de datos

Organización y Calidad (15 puntos):

- 5 puntos: Código bien organizado en funciones
- 3 puntos: Uso apropiado de constantes
- 3 puntos: Nombres descriptivos y comentarios útiles
- 2 puntos: Manejo del módulo `datetime`
- 2 puntos: Implementa buenas prácticas de respaldo

5.3. Penalizaciones

- **-5 puntos:** No usar gestor de contexto `with` para archivos
- **-5 puntos:** No manejar encoding UTF-8 correctamente
- **-3 puntos:** No validar datos antes de guardar
- **-3 puntos:** Código sin organización en funciones
- **-3 puntos:** No usar el módulo `csv` (procesar manualmente)
- **-2 puntos:** No manejar excepciones de archivo
- **-2 puntos:** Variables o funciones con nombres no descriptivos
- **-2 puntos:** No incluir comentarios explicativos
- **-5 puntos:** Plagio o código copiado sin comprensión

6. Desafíos Opcionales (Puntos Extra)

6.1. Nivel Intermedio (+10 puntos cada uno)

1. Importar desde CSV externo

- Permitir importar contactos desde otro archivo CSV
- Validar formato y detectar duplicados
- Opción de fusionar o reemplazar contactos existentes

2. Exportar a formato JSON

- Implementar exportación a formato JSON
- Usar módulo `json`
- Mantener estructura de datos adecuada

3. Búsqueda avanzada

- Búsqueda con múltiples criterios simultáneos
- Operadores lógicos (AND, OR)
- Ordenar resultados por diferentes campos

6.2. Nivel Avanzado (+15 puntos cada uno)

4. Sistema de registro de cambios (log)

- Crear archivo de log que registre todas las operaciones
- Incluir timestamp, usuario (nombre del equipo), acción realizada
- Usar módulo `logging`

5. Manejo de fotografías

- Permitir asociar una foto a cada contacto
- Almacenar ruta de la imagen en el CSV
- Validar que el archivo de imagen exista
- Crear carpeta de imágenes organizada

6. Cifrado básico de datos

- Implementar cifrado simple de emails o teléfonos sensibles
- Usar módulo `base64` o similar
- Agregar opción de cifrar/descifrar

6.3. Nivel Experto (+20 puntos)

7. Sincronización con múltiples archivos

- Mantener versiones de respaldo automáticas
- Detectar y resolver conflictos de datos
- Implementar sistema de versionado

8. Interfaz gráfica básica

- Crear interfaz gráfica simple con `tkinter`
- Mantener todas las funcionalidades del sistema
- Mejorar experiencia de usuario

7. Consejos y Recomendaciones

7.1. Para el Desarrollo

1. **Comienza simple:** Implementa primero agregar y listar, luego añade funcionalidades
2. **Prueba frecuentemente:** Verifica cada función antes de continuar
3. **Usa datos de prueba:** Crea contactos variados para probar todos los casos
4. **Maneja errores:** Anticipa qué puede salir mal y manéjalo
5. **Versiona tu código:** Guarda versiones de respaldo mientras desarrollas
6. **Documenta mientras programas:** No dejes la documentación para el final

7.2. Para la Defensa

1. **Conoce tu código:** Entiende cada línea que escribiste
2. **Practica explicaciones:** Ensaya cómo explicarías conceptos clave
3. **Prepara el ambiente:** Ten tu código listo para ejecutar y mostrar
4. **Anticipa preguntas:** Piensa qué preguntarías tú sobre tu código
5. **Sé honesto:** Si no sabes algo, reconócelo y muestra disposición a aprender
6. **Muestra ejemplos:** Ten casos de prueba preparados para demostrar funcionalidad

7.3. Recursos de Apoyo

- Documentación oficial de Python: <https://docs.python.org/3/>
- Tutorial de archivos: <https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files>
- Módulo CSV: <https://docs.python.org/3/library/csv.html>
- Módulo OS: <https://docs.python.org/3/library/os.html>
- Módulo datetime: <https://docs.python.org/3/library/datetime.html>

8. Criterios de Entrega

8.1. Archivos a Entregar

1. **agenda_contactos.py**: Código principal del programa
2. **README.txt**: Instrucciones de uso y descripción
3. **contactos.csv**: Archivo de ejemplo con al menos 5 contactos de prueba
4. **requirements.txt**: Dependencias (si usas librerías adicionales)

8.2. Formato del README.txt

Tu archivo README debe incluir:

- Nombre del estudiante
- Descripción breve del proyecto
- Instrucciones de ejecución
- Funcionalidades implementadas
- Desafíos opcionales completados (si aplica)
- Problemas conocidos o limitaciones
- Recursos utilizados para el desarrollo

8.3. Evaluación de la Defensa

La defensa será individual y consistirá en:

1. **Demostración (5 minutos)**: Ejecutar el programa y mostrar funcionalidades
2. **Explicación de código (10 minutos)**: Explicar secciones específicas del código
3. **Preguntas conceptuales (5 minutos)**: Responder preguntas sobre conceptos teóricos
4. **Resolución de problemas (5 minutos)**: Modificar el código para agregar una pequeña funcionalidad

9. Notas Finales

- **Fecha de entrega**: Según calendario del curso
- **Peso de la tarea**: 10% de la nota final del curso
- **Nota mínima**: 60/100 para aprobar esta tarea
- **Consultas**: Usar los canales oficiales del curso
- **Integridad académica**: El código debe ser 100% propio. Plagios serán penalizados con 0.

*”Los datos son el nuevo petróleo, pero a diferencia del petróleo,
los datos no se agotan cuando se usan... si sabes cómo almacenarlos correctamente.”*

— Adaptación libre sobre la importancia de la persistencia de datos

¡Éxito en tu desarrollo!

Recuerda: Un buen sistema de archivos es la base de cualquier aplicación seria.